

Individual Contribution Report

CMPE 321 -- Introduction to Database Systems -- Project 3

Personal Information

Name: Beratcan Dogan

Student ID: 2021400132

Group Number: 7

Project: Modular DBMS Engine (four-layer Python implementation)

Tasks & Contributions

Our group divided the project as a continuous pair-programming effort rather than splitting modules between individuals. Every design decision and code change was discussed and reviewed by both team members; our individual contributions are therefore equal in scope and roughly symmetric.

On the architecture side, I worked on shaping the four-layer skeleton, the Result dataclass contract in results.py, and the design decisions document. I drafted the Disk Space Manager (file-header page, logical-to-physical page translation, free-list bookkeeping, I/O counters, the log_write stub) and co-implemented the Buffer Manager, where I focused on the OrderedDict-based LRU/MRU shared structure and the pin/unpin protocol. For Layer 3 I led the slotted-page byte routines and the system catalog (persistent type metadata via a paged file). I also wrote the hash index (static hashing on the primary key with overflow chaining) and the workload generator.

On the experimentation side, I set up the runner script that automates the three experiments, parses stats_output.txt, and emits Markdown tables. The discussion of sequential flooding in Experiment 1 and the buffer-pool sensitivity discussion in Experiment 3 were authored jointly; I drafted the prose for Experiment 1 and the closing notes section of the report.

Collaboration & Challenges

The team coordinated through regular working sessions, mostly over screen-share so that both members could see and react to every code change in real time. We used a shared Git repository with frequent small commits, which let either of us continue the other's work without merge conflicts.

The most demanding challenge was deciding the byte-level formats early on -- record widths, page header layout, free-space tracking, catalog page format -- because a poor choice at this stage would have rippled through every layer. We resolved this by writing the DESIGN.md document before any non-trivial code, so both teammates were operating against the same explicit contract. A second challenge was a B+ tree bug where new internal root pages were allocated but never properly written, tripping an assertion only because we had assertions enabled; tracking it down reinforced the value of leaving sanity checks in even when they look noisy.

Self-Assessment

Working through this project gave me a concrete understanding of how a database engine layers responsibilities: I had only studied buffer-pool replacement and B+ trees abstractly before, and seeing the cache-hit vs cache-miss numbers actually shift with each configuration change made the theory click. I am most satisfied with the cleanliness of the cross-layer contract; swapping LRU for MRU or heap_scan for bplus_tree without editing any other module was a strong signal that the abstraction worked. If I were to do this again, I would start the experiment-running script earlier -- once it existed, iterating on the engine became much faster because every change produced a regression-free table within seconds.

Use of AI Tools

We used Anthropic's Claude (Claude Opus 4.7, via claude.ai) as a design partner and code-review aid throughout the project; full details are in `ai_usage.md`. We discussed each design decision with the assistant before writing code, asked it to draft scaffolding and tests, and consulted it when debugging non-obvious failures. We never pasted output without reading and editing it, and we can explain every line of the submitted code.